

MediClaim Orchestrator

# Multi-Agent Cashless and Reimbursement Claim Processing

Project Report

---

A hospital system and an insurer / TPA system that process a health-insurance claim end to end with CrewAI agents, n8n workflows and deterministic engines, on local open models

Repository: [github.com/Multi-Agent-MediClaim-Orchestrator/Sohail-Ahmed](https://github.com/Multi-Agent-MediClaim-Orchestrator/Sohail-Ahmed)

Prepared by: the project team

October 2026 · Version 1.0

## Abstract

Cashless and reimbursement health-insurance claims in India still move between hospitals and insurers as bundles of scanned documents that people read, check and re-key. A discharge can wait hours for approval, and errors on either side lead to queries, short payments and rejections. This project builds two independent, cooperating systems that automate the work while keeping people in charge of every decision that matters.

On the **hospital side**, documents are scanned for viruses, read with OCR, classified, masked for personal data and turned into structured fields; the claim is assembled from the bills, checked against twelve validation rules, and given an **estimate of the admissible amount** computed from the patient's own policy wording before it is submitted. On the **insurer side**, seven AI agents verify identity, authenticity and coverage, map bill lines and summarise the case, a deterministic engine computes the payable amount, and a decision gate routes the claim to automatic approval, one approver or two approvers. Queries, replies, decisions and settlement flow between the two systems through a signed API, and both keep a tamper-evident audit trail.

The agents are built with **CrewAI** (Agents, Tasks, Crews and Flows with guardrails), the business processes run on **n8n**, and every number and gate is decided by tested code. All models run locally through **Ollama** (gemma4 and nomic-embed-text), so no patient data leaves the machine and no paid API keys are needed. The whole system starts and runs with one command (**make demo**). On a MacBook Air it passed end to end on the real model, and the offline variant passed all seven claim scenarios; the evaluation on 300 synthetic cases shows complete agreement of the calculation engine with an independent reference and no wrongful automatic approvals.

## Contents

|                                                      |    |
|------------------------------------------------------|----|
| Abstract.....                                        | 2  |
| 1. Executive summary.....                            | 4  |
| 2. Background: health-insurance claims in India..... | 5  |
| 2.1 Cashless and reimbursement claims.....           | 5  |
| 2.2 Who does what.....                               | 5  |
| 2.3 What makes it hard.....                          | 5  |
| 3. Problem statement and objectives.....             | 6  |
| 3.1 Problem statement.....                           | 6  |
| 3.2 Objectives.....                                  | 6  |
| 3.3 Scope.....                                       | 6  |
| 3.4 Constraints.....                                 | 6  |
| 4. Solution overview.....                            | 7  |
| 4.1 Design principles.....                           | 7  |
| 4.2 Where AI is used, and where it is not.....       | 7  |
| 5. System architecture.....                          | 9  |
| 5.1 Components.....                                  | 9  |
| 5.2 Deployment view: containers and processes.....   | 9  |
| 5.3 Interface rules.....                             | 10 |
| 6. End-to-end claim workflow.....                    | 11 |
| 6.1 Step by step.....                                | 11 |
| 6.2 Claim states.....                                | 12 |
| 6.3 Decision gate.....                               | 12 |
| 7. Multi-agent design with CrewAI.....               | 13 |
| 7.1 CrewAI concepts we use.....                      | 13 |
| 7.2 Hospital crew.....                               | 13 |
| 7.3 Insurer crew.....                                | 14 |
| 7.4 Example: a guardrail at work.....                | 15 |
| 7.5 Offline mode.....                                | 15 |
| 8. Admissible-amount estimate before submission..... | 16 |

|                                                        |    |
|--------------------------------------------------------|----|
| 8.1 Method.....                                        | 16 |
| 8.2 Worked example.....                                | 16 |
| 9. Workflow automation with n8n.....                   | 17 |
| 9.1 Screenshots of the real workflows.....             | 17 |
| 10. Document intelligence and knowledge retrieval..... | 20 |
| 10.1 doc-pipeline.....                                 | 20 |
| 10.2 vision-service.....                               | 20 |
| 10.3 rag-service.....                                  | 20 |
| 11. Calculation engine and validation rules.....       | 21 |
| 12. Data, privacy, security and audit.....             | 22 |
| 12.1 Synthetic data.....                               | 22 |
| 12.2 Privacy (DPDP-minded).....                        | 22 |
| 12.3 Security.....                                     | 22 |
| 12.4 Audit.....                                        | 22 |
| 13. One-command deployment.....                        | 23 |
| 14. Testing and evaluation.....                        | 25 |
| 14.1 Automated tests.....                              | 25 |
| 14.2 End-to-end scenarios.....                         | 25 |
| 14.3 Evaluation.....                                   | 25 |
| 15. Results.....                                       | 26 |
| 15.1 Real-model run.....                               | 26 |
| 15.2 Offline run.....                                  | 26 |
| 15.3 Observations.....                                 | 26 |
| 16. Limitations.....                                   | 27 |
| 17. Future recommendations and roadmap.....            | 28 |
| 17.1 Short term (next 4-6 weeks).....                  | 28 |
| 17.2 Medium term (3-6 months).....                     | 28 |
| 17.3 Long term.....                                    | 28 |
| 18. Conclusion.....                                    | 29 |
| References.....                                        | 30 |
| Appendix A. Glossary.....                              | 31 |
| Appendix B. Repository structure.....                  | 32 |
| Appendix C. Command reference.....                     | 33 |
| Appendix D. Agent service endpoints.....               | 34 |
| Appendix E. More n8n workflow screenshots.....         | 35 |
| Appendix F. Design decisions log (summary).....        | 37 |

# 1. Executive summary

**What we built.** Two systems, a hospital and an insurer / TPA, that together process a health-insurance claim from the first uploaded document to the bank payout. Each side has its own API, database, workflows, AI agents and user interface, and they talk only through a versioned, signed REST contract.

**Why.** The manual process is slow (hours from discharge to approval), error-prone (arithmetic, missing documents, wrong categories) and opaque (the hospital does not know what the insurer will pay until it pays). Our design targets all three: automation for speed, deterministic checks for correctness, and quoted evidence plus an estimate for transparency.

**How.** AI agents do what only language models can do well: read unstructured text, map free-text bill lines, find and quote policy clauses, and draft polite, grounded messages. Code does everything that must be exact: arithmetic, policy rules, identity matching, routing and the final money. Humans approve anything above ₹50,000, and two approvers anything above ₹5,00,000.

| Area           | Outcome                                                                                                                 |
|----------------|-------------------------------------------------------------------------------------------------------------------------|
| Agents         | 11 CrewAI agents (4 hospital, 7 insurer) in 4 CrewAI Flows; every model call guarded                                    |
| Workflows      | 32 n8n workflows (13 hospital, 19 insurer)                                                                              |
| One command    | <b>make demo</b> (real model) and <b>make demo-offline</b> (no model) bring up everything and run a claim end to end    |
| Real-model run | Passed on a MacBook Air (Apple Silicon): claim built, verified, auto-approved, settled, audit verified; 166 s claim run |
| Scenarios      | 7/7 passed offline: auto, reviewer, dual approval, rejection, three query rounds, duplicate callbacks, bank retry       |
| Quality        | 1,000+ automated tests; calculation agreement 1.00; wrongful automatic approvals 0.00                                   |
| Privacy        | Local models only, PII masked before any model call, no paid keys, synthetic data only                                  |

The remaining gaps are clearly bounded: retrieval citation precision (0.61 against a 0.90 target, measured offline), a few success metrics not yet measured (manual baseline, time to submit, queries per claim), and moving the insurer's step order fully into the CrewAI Flow. Section 17 lays out the plan.

## 2. Background: health-insurance claims in India

### 2.1 Cashless and reimbursement claims

A **cashless** claim is settled directly between a network hospital and the insurer (often through a Third-Party Administrator, TPA): the patient pays only what the policy does not cover. A planned cashless admission needs a pre-authorisation; at discharge the hospital submits the final claim. A **reimbursement** claim is paid to the patient after they settle the bill themselves and submit the documents. The same verification applies to both; only the payee and the timing differ.

### 2.2 Who does what

| Party                      | Role in a claim                                                                                                |
|----------------------------|----------------------------------------------------------------------------------------------------------------|
| Patient / member           | Holds the policy; provides identity proof and the policy card at admission                                     |
| Hospital billing desk      | Collects bills, discharge summary, prescriptions and reports; assembles and submits the claim; answers queries |
| Billing officer            | Reviews the assembled claim and signs it off before submission                                                 |
| Insurer / TPA reviewer     | Checks identity, authenticity, coverage and the amounts; raises queries; recommends a decision                 |
| Approver / senior reviewer | Approves or rejects above set amounts                                                                          |
| Bank                       | Pays the settled amount to the hospital (cashless) or the member (reimbursement)                               |

### 2.3 What makes it hard

- **Unstructured, noisy input.** Scanned or photographed documents with stamps, handwriting, skew and low resolution.
- **Policy complexity.** Room-rent caps with proportional deductions on related charges, ICU caps, procedure sub-limits, co-payments, deductibles, waiting periods, exclusions and non-payable items all interact, and the wording changes by product version and date.
- **Back-and-forth.** Missing or unclear documents lead to queries; each round can take a day; after three rounds the case escalates.
- **Trust and privacy.** Claims contain identity numbers and diagnoses. India's Digital Personal Data Protection Act, 2023 (DPDP) sets duties on how such data is processed.

## 3. Problem statement and objectives

### 3.1 Problem statement

#### Problem

Build a multi-agent system that takes a hospital's claim from raw documents to a submission-ready claim, verifies and decides it on the insurer's side, and handles queries and settlement, so that discharge-to-approval time drops from hours to minutes while every decision above a threshold stays with a human, every step is auditable, and no patient data leaves the organisation.

### 3.2 Objectives

1. Automate document intake: virus scan, OCR, classification, extraction and PII masking.
2. Assemble the claim from the bills and validate it before a person sees it.
3. Interpret the policy on the hospital side (sub-limits, co-pays, non-payables) and estimate the admissible amount before submission.
4. Verify the claim on the insurer side with specialised agents, compute the payable amount exactly, and route the decision by risk and amount.
5. Run the query loop (raise, triage, grounded reply, escalate) and settlement end to end.
6. Use CrewAI for the agents and n8n for the business workflows, as required by the course.
7. Start and run everything with a single command, and measure the results.

### 3.3 Scope

| In scope                                                           | Out of scope                                           |
|--------------------------------------------------------------------|--------------------------------------------------------|
| Cashless and reimbursement; planned and emergency admissions       | Real pre-authorisation (a simulated record is assumed) |
| Document intake, completeness, claim build and submission          | Real insurer, TPA, bank, ABDM or NHCX integrations     |
| Identity, authenticity, coverage, calculation, decision gate       | Real patient data, production hosting, commercial use  |
| Multi-round query loop with escalation; settlement via a simulator | Fraud analytics across insurers                        |
| Admin configuration, audit explorer, evaluation harness            | Mobile apps                                            |

### 3.4 Constraints

- **No paid API keys and no patient data leaving the machine:** all models run locally through Ollama.
- **Commodity hardware:** one laptop, 16 GB RAM minimum, no GPU required.
- **Synthetic data only:** members, policies, hospitals and documents are generated, with known ground truth for evaluation.

## 4. Solution overview

The solution is two systems that mirror the real world: the hospital and the insurer are separate organisations with separate data. Each system owns its API, database, workflow engine, agents and user interface. The only link is a versioned REST contract with HMAC signatures and idempotency keys, plus signed callbacks in the other direction.

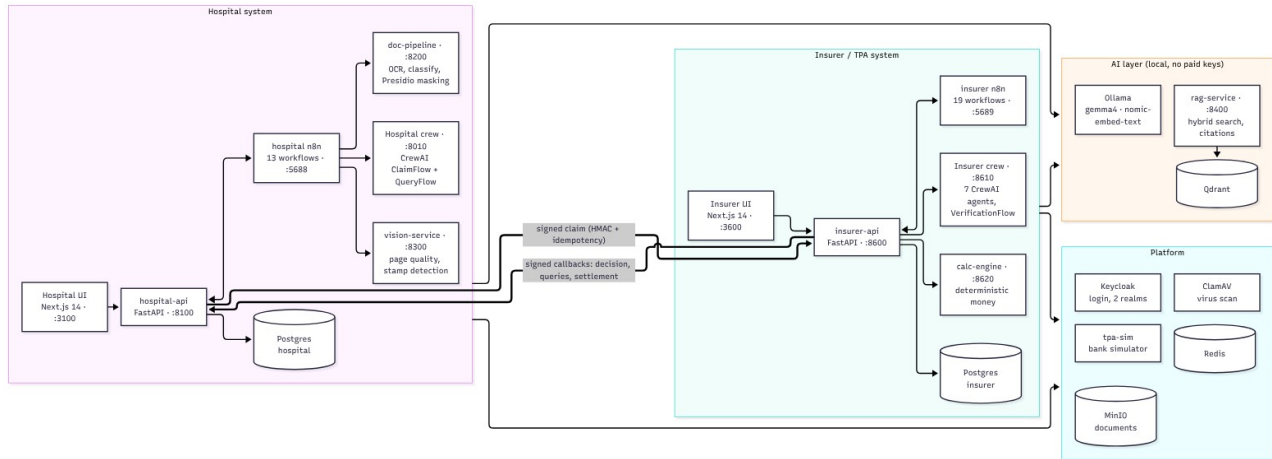


Figure 1: System architecture: hospital system, insurer / TPA system, AI layer and platform services

### 4.1 Design principles

| Principle                | What it means in the code                                                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Deterministic first      | Routing, completeness, identity matching, arithmetic and payout are code. Models only read text, map lines and draft messages; code checks and can override their answers. |
| Configuration over code  | Document requirements, thresholds, deadlines and policy rules are versioned database rows; each decision records the versions it used.                                     |
| Human authority          | The system recommends; people decide above ₹50,000, and two people above ₹5,00,000. Any blocker sends a claim to a person.                                                 |
| Privacy by construction  | Identity documents never go to a model; text is masked first; a PII guard runs before every model call; models are local.                                                  |
| Evidence over confidence | Gates use parser confidence, quotes and code checks, never a model's self-reported confidence.                                                                             |
| Everything auditable     | Every agent output, human edit and approval is an append-only, hash-chained event on both sides.                                                                           |
| Fixable before fatal     | A missing document or stamp becomes a needs-info request, not a rejection.                                                                                                 |

### 4.2 Where AI is used, and where it is not

| Task                                    | Done by                                  | Why                                                      |
|-----------------------------------------|------------------------------------------|----------------------------------------------------------|
| OCR, classification, field extraction   | doc-pipeline (rules + model, two passes) | Unstructured scans; two independent passes must agree    |
| Mapping unknown bill lines              | CrewAI agents (hospital and insurer)     | Free text; code limits answers to the allowed categories |
| Reading policy wording, quoting clauses | CrewAI agents + RAG                      | Long natural-language documents; quotes are verified     |
| Drafting queries and replies            | CrewAI agents                            | Natural language; grounding                              |

| Task                                  | Done by            | Why                                 |
|---------------------------------------|--------------------|-------------------------------------|
|                                       |                    | guardrails reject invented facts    |
| Arithmetic, caps, co-pay, sub-limits  | calc-engine (code) | Must be exact and reproducible      |
| Identity match, completeness, routing | Code               | Rules with known answers; auditable |
| Approve / reject above thresholds     | People             | Accountability                      |

## 5. System architecture

### 5.1 Components

Table 1: Main components

| Component      | Technology                                          | Port    | Responsibility                                                                           |
|----------------|-----------------------------------------------------|---------|------------------------------------------------------------------------------------------|
| hospital-ui    | Next.js 14                                          | 3100    | Desk, officer and admin screens; live status                                             |
| hospital-api   | FastAPI, SQLAlchemy, Alembic                        | 8100    | Cases, documents, completeness, router, claim builder, submission outbox, queries, audit |
| hospital n8n   | n8n 2.41.6                                          | 5688    | 13 workflows: intake, completeness, build, submission, queries, reminders                |
| hospital crew  | CrewAI 1.15.23                                      | 8010    | ClaimFlow and QueryFlow: claim build/repair, estimate, triage, grounded replies          |
| doc-pipeline   | Poppler, Tesseract, Presidio                        | 8200    | Parse, classify, mask, two-pass extraction                                               |
| vision-service | OpenCV, NumPy                                       | 8300    | Page quality, stamp detection, registry match                                            |
| insurer-ui     | Next.js 14                                          | 3600    | Reviewer, approver and admin workspace                                                   |
| insurer-api    | FastAPI, SQLAlchemy, Alembic                        | 8600    | Receipt, verification engine, decision gate, queries, settlement                         |
| insurer n8n    | n8n 2.41.6                                          | 5689    | 19 workflows: verification steps, decision gate, queries, escalation, settlement         |
| insurer crew   | CrewAI 1.15.23                                      | 8610    | 7 agents; VerificationFlow and QueryFlow                                                 |
| calc-engine    | Python (Decimal)                                    | 8620    | Payable amount; used in process by insurer-api and as a library by the hospital          |
| rag-service    | Own hybrid search on Qdrant                         | 8400    | Policy wording, guidelines, precedents; filtered and cited retrieval                     |
| tpa-sim        | FastAPI                                             | 8500    | Bank / TPA simulator for payouts and scripted queries                                    |
| Ollama         | gemma4, nomic-embed-text                            | 11434   | Local language and embedding models                                                      |
| Platform       | Postgres 16 ×2, Redis 7, MinIO, ClamAV, Keycloak 25 | various | Data, queues, documents, virus scan, login                                               |

### 5.2 Deployment view: containers and processes

`make demo` runs the stateful platform in Docker containers and the Python services as host processes (faster to restart and easier to debug on a laptop). Three Docker networks keep data apart: the hospital database is reachable only on `hospital_net`, the insurer database only on `insurer_net`, and the shared services sit on `shared_net` as well as both private networks. The two n8n containers use host networking so they can call the host services.

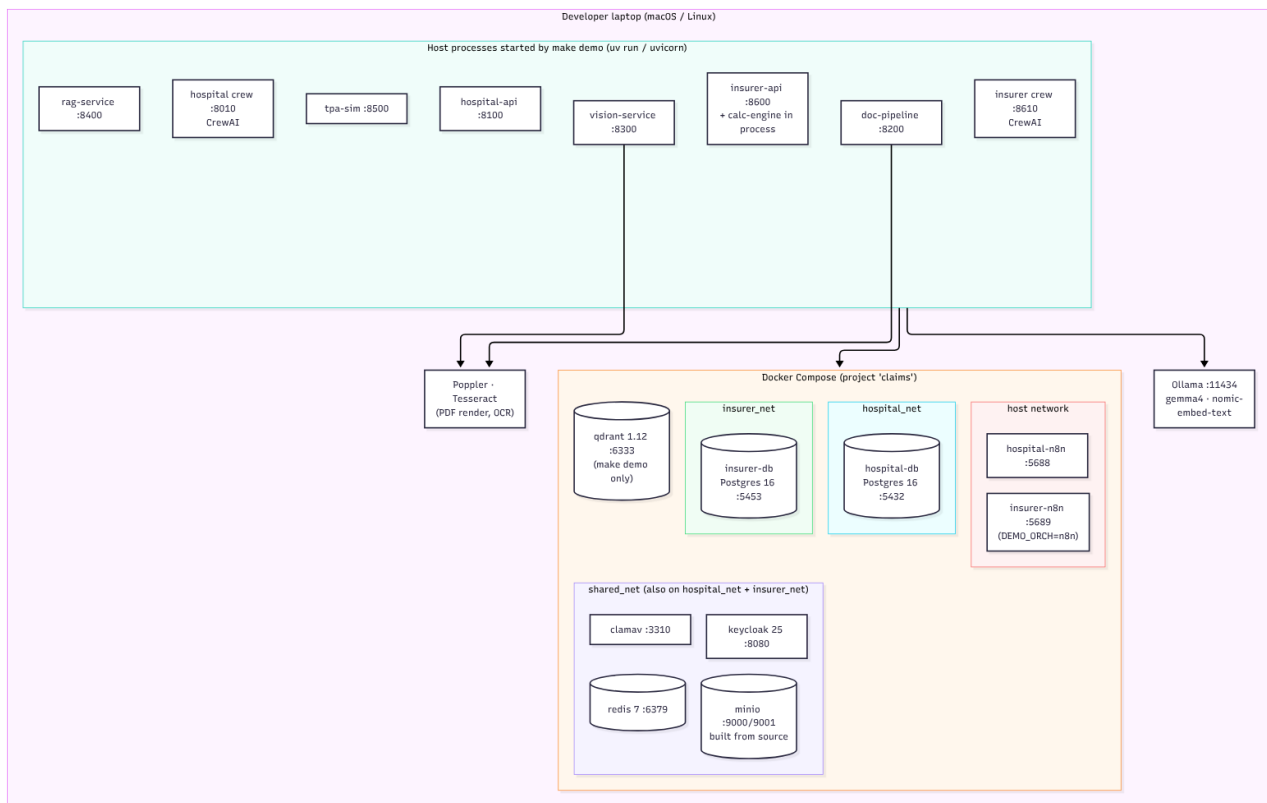


Figure 2: Deployment view: Docker containers by network, host processes, Ollama and the PDF/OCR tools

### 5.3 Interface rules

- UIs call only their own API; the two APIs never share a database.
- Hospital ↔ insurer traffic goes only through the versioned REST contract, signed with HMAC and carrying an idempotency key; callbacks are signed the same way.
- n8n calls the APIs with a service account; the crews never write to a database: they return validated objects that the API checks and stores.
- The hospital's knowledge-base token can read only **hosp\_\*** collections; the insurer's collections are closed to it.

## 6. End-to-end claim workflow

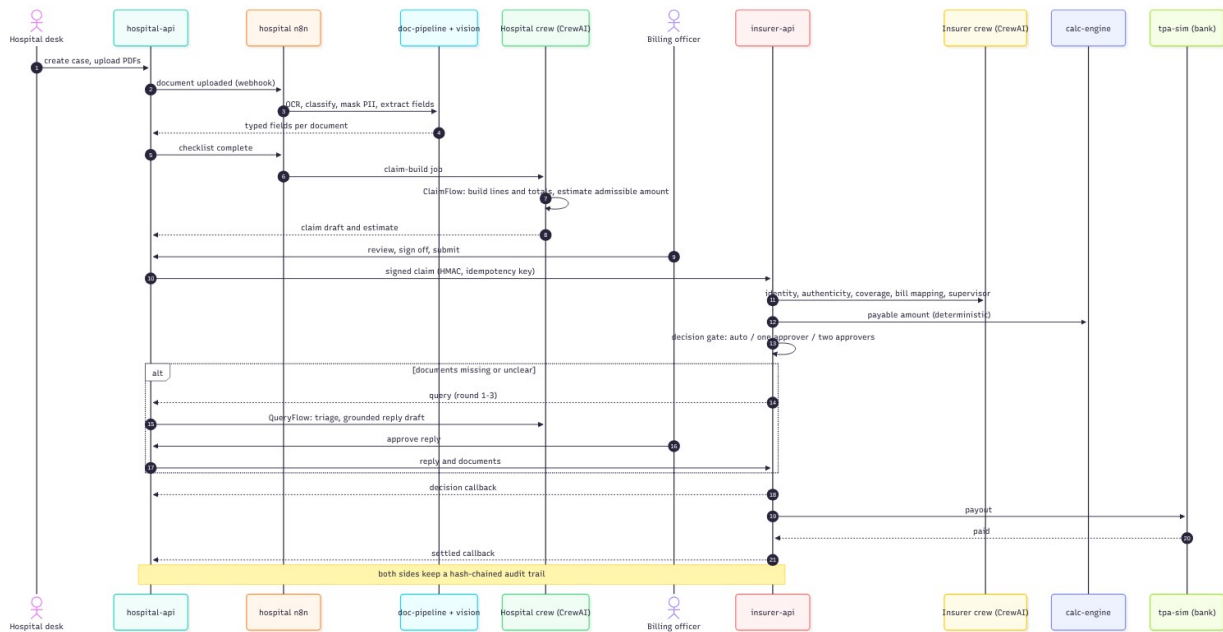


Figure 3: Claim lifecycle from upload to settlement (sequence of calls)

### 6.1 Step by step

- Intake.** The desk creates a case and uploads PDFs. hospital-api checks size and type, scans each file with ClamAV, stores it encrypted in MinIO and triggers the n8n intake workflow.
- Reading.** n8n calls vision-service (page quality, stamps) and doc-pipeline (text layer or OCR, classification, Presidio masking, two-pass field extraction). Low-confidence pages go to review.
- Completeness.** hospital-api compares the documents with the configured checklist (for example prescription, pharmacy bill, final bill, plus procedure bill or implant sticker when relevant) and asks for what is missing.
- Claim build (CrewAI ClaimFlow).** The hospital crew assembles bill lines and totals in code, asks the category-mapper agent only for unknown lines, then estimates the admissible amount from the policy wording and the calculation engine.
- Validation and sign-off.** hospital-api applies validation rules V01-V12 (totals, quantity × rate, dates, identity, ICD-10 codes, duplicates, pre-auth excess, room-rent cap, document integrity). The billing officer reviews, edits if needed, sees the estimate and signs off.
- Submission.** A signed, idempotent submission goes to insurer-api through an outbox with retries.
- Verification.** insurer-api runs document fetch, completeness, identity, authenticity, coverage and calculation. Each AI step calls its CrewAI agent; code computes the facts and the money.
- Decision gate.** A clean claim of ₹50,000 or less is approved automatically; above that one approver decides; above ₹5,00,000 two approvers. Blockers, warnings or degraded steps always send it to a person.
- Queries.** If something is missing or unclear, the insurer's query drafter writes the query; the hospital's QueryFlow triages it and drafts a grounded reply for the officer to approve. After three rounds the case escalates to a senior reviewer.
- Settlement and audit.** The decision is sent back by signed callback; the payout goes through the bank simulator; both audit chains are verified.

## 6.2 Claim states

| Hospital case states                                                                                                                                                            | Insurer case states                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| draft → docs_pending → docs_complete → building_claim → ready_for_review → submitted → acknowledged → under_query → approved / partially_approved / rejected → settled → closed | received → verifying → needs_info → ready_for_decision → awaiting_approval → approved / partially_approved / rejected / escalated → settled → closed |

## 6.3 Decision gate

Table 2: Decision tiers (insurer-api gate.py)

| Tier                       | When                                                                                                                        | Who decides                           |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| auto                       | Clean claim, amount $\leq$ T_auto (₹50,000), identity score above threshold, no blockers, warnings, flags or degraded steps | The system (recorded with the reason) |
| reviewer / single approver | Amount > T_auto, or any flag or warning                                                                                     | One reviewer or approver              |
| dual approver              | Amount > T_four (₹5,00,000), or rejection of a large claim                                                                  | Approver and senior reviewer          |

## 7. Multi-agent design with CrewAI

### 7.1 CrewAI concepts we use

| Concept    | Meaning                                                                                 | Where in our code                                                              |
|------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Agent      | A role, a goal and a backstory, bound to a model                                        | <code>config/agents.yaml</code> on both sides                                  |
| Task       | One piece of work with an expected output                                               | One task per model call ( <code>team.py</code> , <code>crewai_team.py</code> ) |
| Crew       | Agents and tasks run in a process (sequential here)                                     | A one-task sequential crew per call                                            |
| Flow       | Owns state and order: <code>@start</code> , <code>@listen</code> , <code>@router</code> | ClaimFlow, QueryFlow (hospital);<br>VerificationFlow, QueryFlow (insurer)      |
| Guardrail  | Checks a task's output; a failure is sent back to the agent                             | Grounding rules, quote checks, schema checks                                   |
| Custom LLM | A <code>BaseLLM</code> subclass that CrewAI calls                                       | BridgeLLM / GatewayBridge: our own clients behind CrewAI                       |

Every agent reaches the model through a small bridge, so the protections that existed before CrewAI stay in front of every call: the PII guard, the Ollama circuit breaker, JSON repair, token accounting, the degraded-model flag and Langfuse tracing. CrewAI's memory and knowledge features are switched off (they default to OpenAI embeddings) and telemetry is opted out. Agents have no tools: deterministic tools run in the flow steps and as guardrails, because small local models are unreliable at multi-step tool use.

### 7.2 Hospital crew

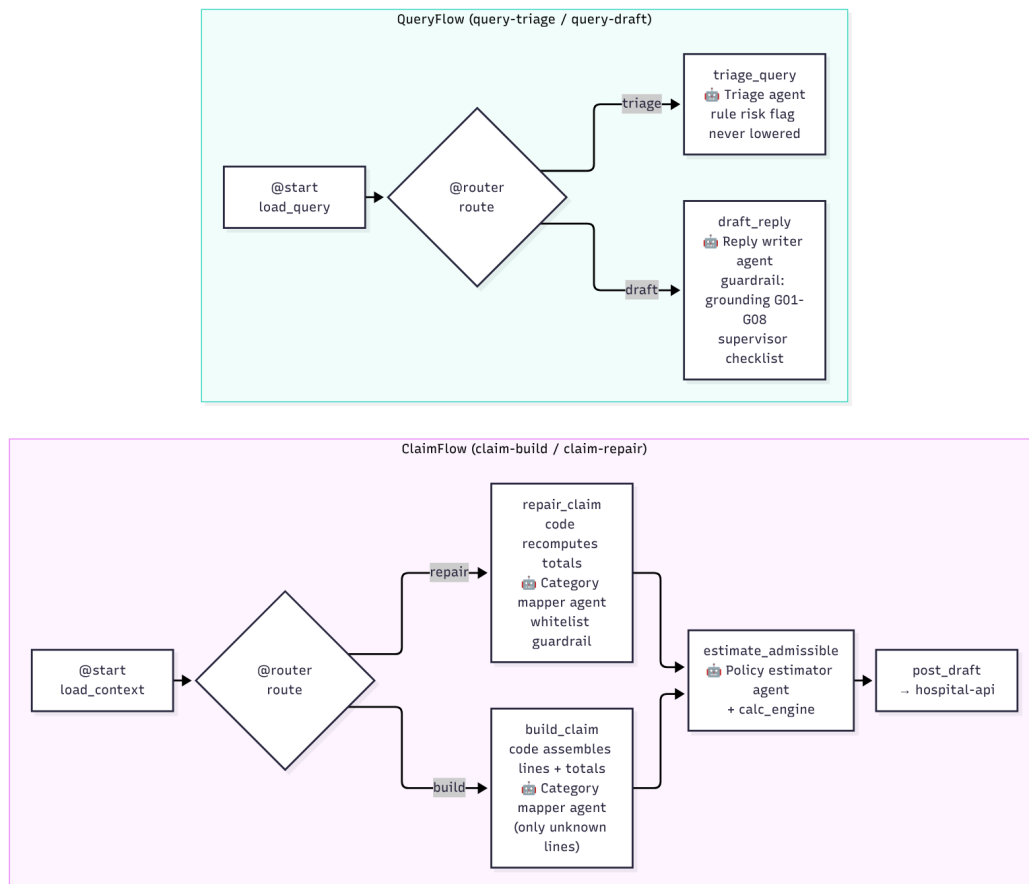


Figure 4: Hospital CrewAI flows: ClaimFlow (build, repair, estimate) and QueryFlow (triage, draft)

| Agent                         | Input → output                                                           | Guardrail / code control                                                   |
|-------------------------------|--------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Category mapper (local model) | Unlabelled bill lines → category per line                                | Only allowed categories accepted; others ignored                           |
| Policy estimator              | Policy wording excerpts → room %, ICU %, co-pay %, sub-limit with quotes | Quote must be in the wording and contain the number; code parser overrides |
| Query triage officer          | Insurer query → send documents / clarify, escalation risk                | Rule-based risk flag can be raised, never lowered                          |
| Claims desk reply writer      | Numbered evidence → reply with verbatim citations                        | Grounding rules G01-G08, one retry; supervisor checklist                   |

## 7.3 Insurer crew

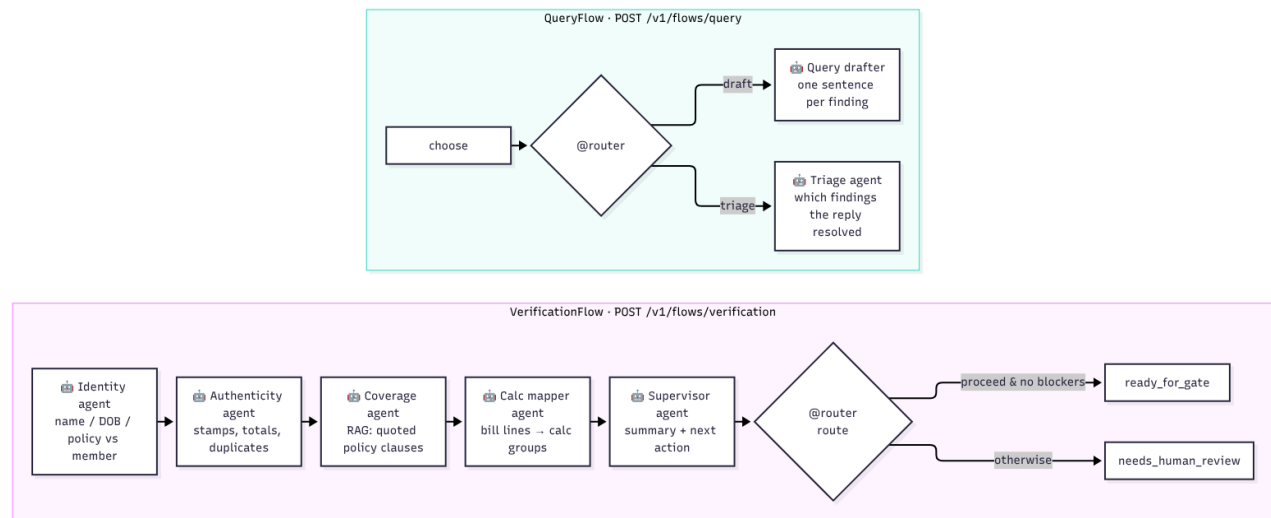


Figure 5: Insurer CrewAI flows: VerificationFlow with routing, and QueryFlow

| Agent         | What it does                                        | What code still decides                                                  |
|---------------|-----------------------------------------------------|--------------------------------------------------------------------------|
| Identity      | Reconciles patient, member and ID-document fields   | Name score, DOB and policy matches are facts it cannot contradict        |
| Authenticity  | Explains stamp, total, duplicate and tamper signals | Anomalies without a code signal or evidence are dropped; severity capped |
| Coverage      | Finds and quotes policy clauses (RAG)               | Quotes must appear in retrieved text; else insufficient evidence         |
| Calc mapper   | Maps unknown lines to calculation groups            | Keyword rules first; calc-engine owns money                              |
| Supervisor    | Summary for the reviewer, next action               | 'Proceed' downgraded if any blocker exists                               |
| Query drafter | One sentence per open finding                       | Forbidden phrases and invented amounts stripped                          |
| Triage        | Which findings a reply resolved                     | Code check of attached documents overrides                               |

The seven agents keep their HTTP endpoints, so insurer-api and the n8n step workflows call them per step. Two Flow endpoints chain them: **POST /v1/flows/verification** (identity → authenticity → coverage → calc mapper → supervisor → proceed or human review) and **POST /v1/flows/query** (draft or triage). A failed step is recorded as a blocker, so the flow routes the claim to a person.

## 7.4 Example: a guardrail at work

### Grounded reply (hospital QueryFlow)

Attempt 1: the agent writes "The room rent was 99999 as recorded in the file."

Guardrail: "number not found in the record: 99999" → sent back to the agent.

Attempt 2: "The room rent of 10000.00 is as billed in the record." with a verbatim quote → accepted; supervisor checklist passes.

## 7.5 Offline mode

With `CREW_LLM=rules` (hospital) and `INS_CREW_LLM=offline` (insurer), the same Flows, agents and guardrails run on deterministic stand-in models that claim nothing on their own. This makes the full pipeline testable in seconds and gives a safe fallback for live demonstrations.

## 8. Admissible-amount estimate before submission

Today a hospital learns what the insurer will pay only after submission. The estimate closes that gap: before sign-off, the officer sees the expected payable amount and the policy terms behind it, quoted from the wording.

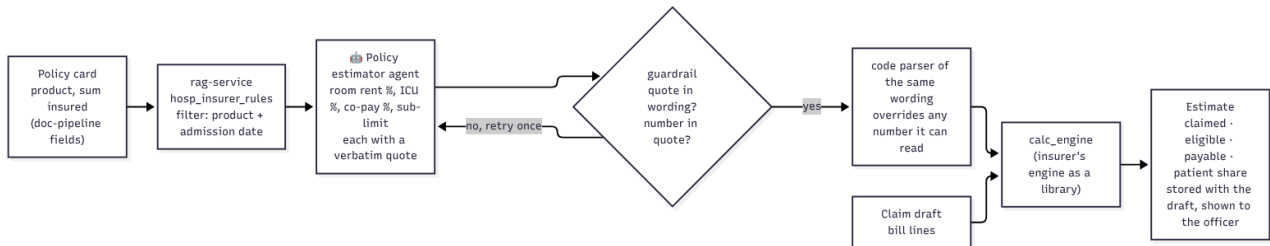


Figure 6: Policy estimate pipeline: policy card, wording, agent with guardrail, code parser, calculation engine

### 8.1 Method

1. Read the product and sum insured from the policy card (doc-pipeline fields `product_name`, `sum_insured`).
2. Retrieve the product's wording from the hospital's knowledge base, filtered by product and the admission date, so the version in force is used.
3. The policy-estimator agent reports room rent %, ICU %, co-pay % and the procedure sub-limit, each with a verbatim quote; the guardrail rejects quotes that are not in the wording or do not contain the number.
4. A code parser reads the same wording; where it can read a term, its number wins (`term_overridden_by_code`).
5. The insurer's calculation engine, used as a library, applies non-payable exclusions, room caps, sub-limits and co-pay to the bill lines.

### 8.2 Worked example

| Item                                                                     | Amount (₹) | Rule                                             |
|--------------------------------------------------------------------------|------------|--------------------------------------------------|
| Claimed (room 4 days × 2,500; appendectomy 45,000; attendant charge 800) | 55,800     |                                                  |
| Attendant service charge                                                 | – 800      | Non-medical, excluded                            |
| Room rent 2,500 / day                                                    | 0          | Below cap: $1.0\% \times 3,00,000 = 3,000$ / day |
| Eligible                                                                 | 55,000     |                                                  |
| Co-payment 10%                                                           | – 5,500    | "A co-payment of 10% applies"                    |
| Estimated payable                                                        | 49,500     | Patient pays 6,300                               |

The estimate is advice, never a gate. Waiting periods and exclusions need the insurer's member history, so the estimate assumes they pass and says so; if the card, the knowledge base or the terms are missing, it records "unavailable" with the reason and the claim continues.

## 9. Workflow automation with n8n

n8n runs the business processes: triggers, webhooks, retries, waits, reminders, SLAs and escalations. The workflows are thin by design: they call the APIs, which hold the rules. They are generated from code (`build_flows.py`), linted, versioned in Git, and imported automatically when the container starts.

**Table 3: Hospital workflows (13)**

| Workflow                                    | Purpose                                                                                                                                                                          |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| hosp-intake-document                        | Per upload: verify webhook secret, idempotency, duplicate check, virus-scan result, vision quality, doc-pipeline parse (two passes), classification, mark needs-review or failed |
| hosp-completeness-loop                      | Recompute the checklist and request missing documents                                                                                                                            |
| hosp-claim-build                            | Start the CrewAI claim-build job; a cron returns stuck builds                                                                                                                    |
| hosp-submission-watch                       | Follow the submission outbox and acknowledgements                                                                                                                                |
| hosp-query-intake / query-draft / query-sla | Receive insurer queries, ask the crew for triage and a draft, enforce the reply SLA                                                                                              |
| hosp-reminders, hosp-sweeper, hosp-error    | In-app reminders, cleanup of stale work, central error handling                                                                                                                  |
| hosp-u1-auth, u2-idem, u3-poll              | Reusable sub-workflows: service token, idempotency, polling                                                                                                                      |

**Table 4: Insurer workflows (19)**

| Workflow             | Purpose                                                                     |
|----------------------|-----------------------------------------------------------------------------|
| 00-04 common         | Error handler, token, API caller, crew caller, event emitter                |
| 10 verification main | Run the steps in order and finalise the run                                 |
| 11-16 steps          | Document fetch, completeness, identity, authenticity, coverage, calculation |
| 20-21 decision       | Decision gate (notify approvers) and final decision                         |
| 30-31 queries        | Query sent and query response handling                                      |
| 40, 50, 60           | Escalation, settlement, SLA and housekeeping cron                           |

### 9.1 Screenshots of the real workflows

The following screenshots are the project's actual workflows in the n8n editor (n8n 2.41.6), laid out with n8n's tidy-up. The red triangles mark nodes whose credentials are configured at runtime by the container.



*Figure 7: Hospital intake workflow (part 1): webhook, secret check, idempotency, duplicate and virus-scan gates, vision quality*

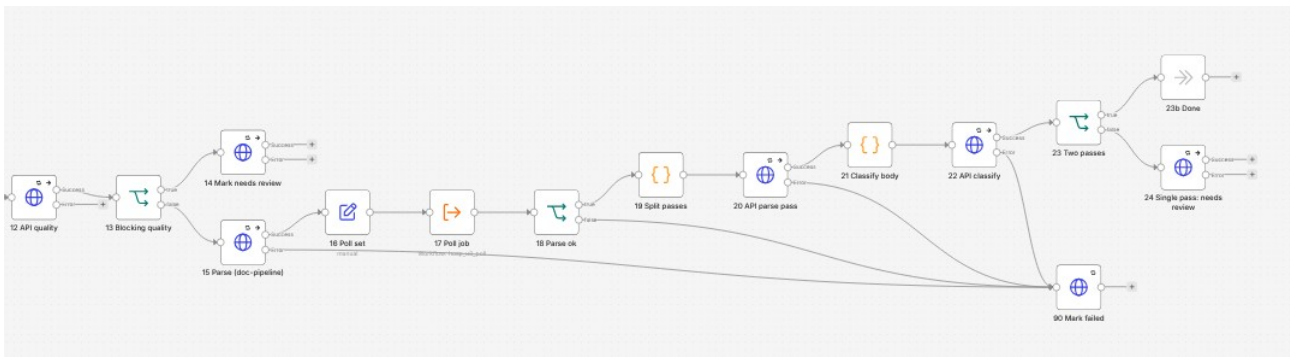


Figure 8: Hospital intake workflow (part 2): doc-pipeline parse, two-pass split, classification, needs-review and failure paths

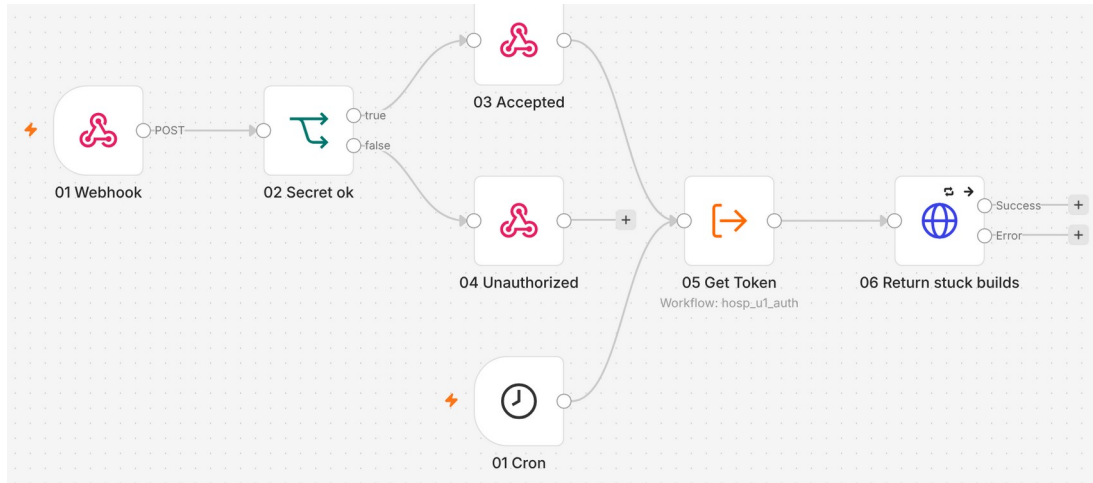


Figure 9: Hospital claim-build workflow: authenticated trigger, token, and the cron that returns stuck builds

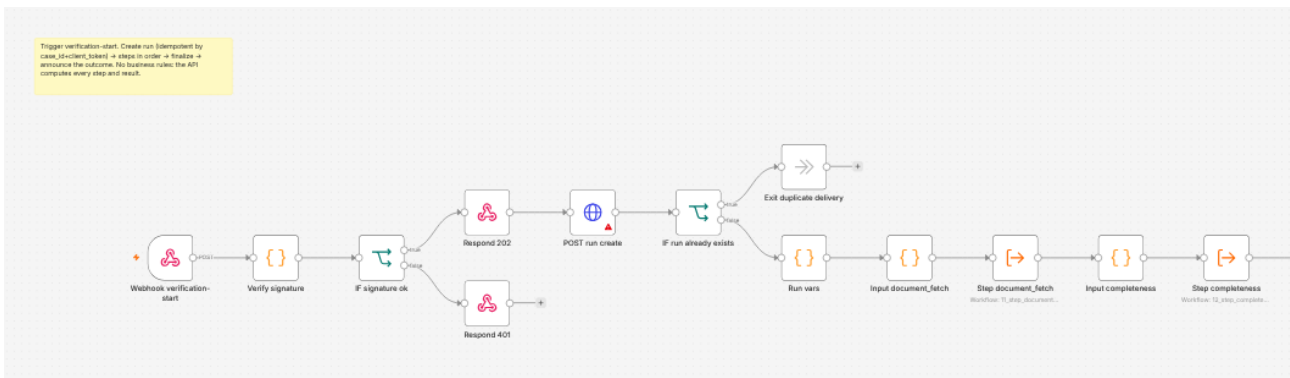


Figure 10: Insurer verification main (part 1): signed webhook, run creation, duplicate-delivery guard



Figure 11: Insurer verification main (part 2): each verification step as a sub-workflow, finalise, decision-ready event

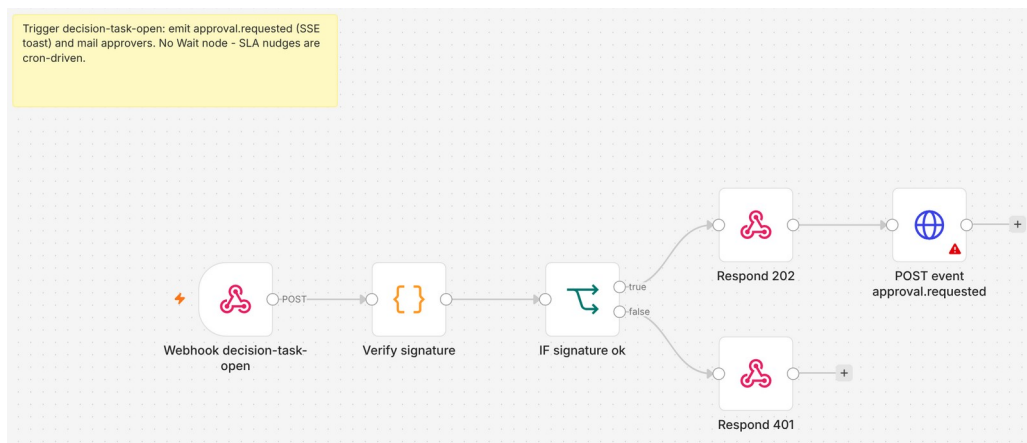


Figure 12: Insurer decision gate: signature check and approval request

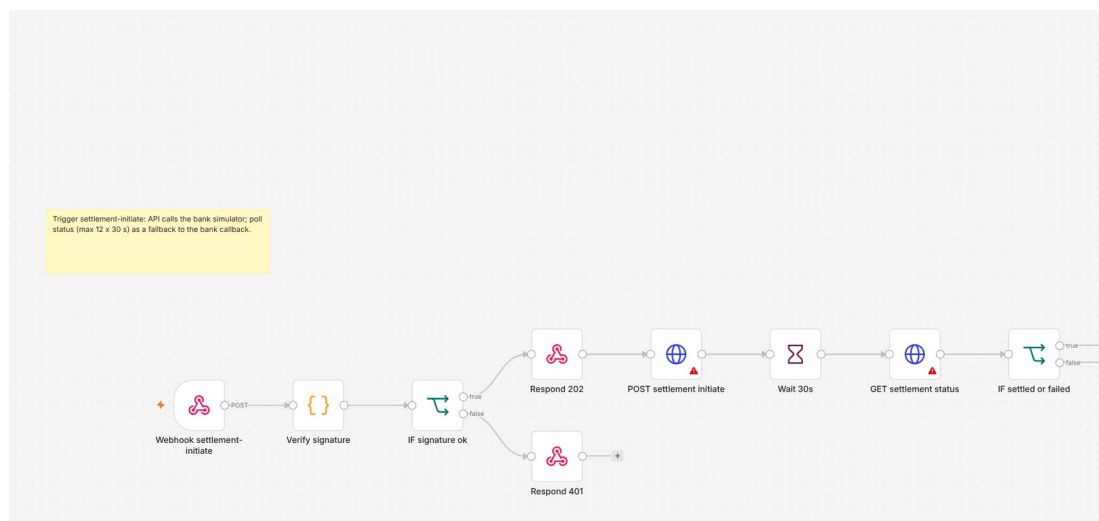


Figure 13: Insurer settlement workflow

## 10. Document intelligence and knowledge retrieval

### 10.1 doc-pipeline

| Stage    | What happens                                                                                                                       |
|----------|------------------------------------------------------------------------------------------------------------------------------------|
| Render   | Text layer with <code>pdf totext</code> ; pages without text are rendered with <code>pdf to ppm</code> and read with Tesseract OCR |
| Classify | Weighted keyword rules per document type (final bill, pharmacy bill, discharge summary, policy card, ...)                          |
| Mask     | Presidio (with spaCy) finds names, phone numbers, Aadhaar, PAN and e-mail; text is masked before any cloud-capable model           |
| Extract  | Typed fields per document type with quotes; two independent passes must agree; a value without supporting text is dropped          |
| Validate | Amounts, dates and critical fields; low parse confidence marks the document for review                                             |

### 10.2 vision-service

Measures page quality (blur, skew, contrast) and detects hospital stamps with a learned detector (a random forest on candidate regions, trained on synthetic stamps; held-out precision/recall 0.99/0.97 against 0.74/0.56 for the classical baseline). Bills must carry a stamp.

### 10.3 rag-service

A retrieval service built for this project: documents are chunked by section, embedded with nomic-embed-text and stored in Qdrant; queries combine dense (vector) and sparse (keyword) search with reranking, filter by product, version and effective date (`as_of`), and return quotable chunks with citation ids. Collections are separated by owner: `ins_policy_wording`, `ins_medical_guidelines`, `ins_case_history` for the insurer, and `hosp_insurer_rules`, `hosp_case_context` for the hospital; tokens are scoped so neither side reads the other's collections.

## 11. Calculation engine and validation rules

The calculation engine is pure Python with **Decimal** arithmetic and no model calls. It takes the policy snapshot, member, admission facts, mapped bill lines and a versioned rule set, and returns per-line results, deductions with reasons, flags and a full trace.

**Table 5: Calculation steps**

| Step                     | Rule                                                                                            |
|--------------------------|-------------------------------------------------------------------------------------------------|
| S0 prechecks             | Policy in force, premium paid (with grace days)                                                 |
| S1 waiting periods       | Initial, pre-existing disease and procedure-specific waiting periods                            |
| S2 exclusions            | Excluded ICD prefixes and tags; non-medical items                                               |
| S3 sub-limits            | Per-procedure limits by sum insured                                                             |
| S4 room rent             | Daily room / ICU cap as % of sum insured; proportional deduction on linked professional charges |
| S5 caps                  | Line caps (e.g. ambulance), pre/post-hospitalisation windows                                    |
| S6 eligible              | Eligible total after the above                                                                  |
| S7-S8 deductible, co-pay | Per-claim deductible; co-pay (age or network conditions)                                        |
| S10 sum insured          | Remaining sum insured caps the payable                                                          |
| S11-S12                  | Allocation back to lines; invariants (no negative amounts, totals reconcile)                    |

**Table 6: Hospital claim validation rules**

| Rule     | Check                                                                     |
|----------|---------------------------------------------------------------------------|
| V01, V02 | Line items add up to gross; gross – discounts = claimed                   |
| V03      | Quantity × unit price = line amount                                       |
| V04      | Each line cites a document and page in this case                          |
| V05      | Admission and discharge dates match the case; lines dated within the stay |
| V06      | Patient name and date of birth match the record                           |
| V07      | At least one ICD-10 diagnosis code                                        |
| V08      | Bill document totals match the claim lines                                |
| V09      | Possible duplicate lines                                                  |
| V10      | Claimed amount over the pre-authorised amount                             |
| V11      | Room rent above the configured daily maximum (warning)                    |
| V12      | Every referenced document belongs to the case and passed the virus scan   |

## 12. Data, privacy, security and audit

### 12.1 Synthetic data

A generator creates members, policies, hospitals and claim cases with realistic bills and documents (ReportLab PDFs, then degraded: blur, skew, noise, stamps, handwriting-like marks), together with the ground truth for every field. Archetypes cover clean claims, missing documents, total mismatches, room-rent overruns, sub-limits, exhausted sum insured and identity mismatches.

### 12.2 Privacy (DPDP-minded)

- Identity documents are never sent to a model; text is masked with Presidio before any model that could leave the machine.
- A regex PII guard (Aadhaar, PAN, phone, e-mail) runs on the final prompt before every model call; a hit fails the job with its own error code.
- All models run locally through Ollama; CrewAI telemetry is opted out; no provider API keys exist in the system (a CI job checks for leaked keys).
- Sensitive columns are encrypted at the application level; documents are stored with server-side encryption.

### 12.3 Security

- Keycloak with two realms (hospital, insurer) and role-based access in both APIs.
- HMAC-signed requests with idempotency keys between the systems; signed callbacks.
- ClamAV virus scan on every upload; size and type checks.
- Prompt-injection defence: document text is wrapped as untrusted data, instruction-like text is neutralised, and a 20-document injection corpus is part of the tests.

### 12.4 Audit

Both sides write an append-only audit trail in which each event carries the hash of the previous one. Any change to a past event breaks the chain. Every end-to-end run verifies both chains as its last step.

## 13. One-command deployment

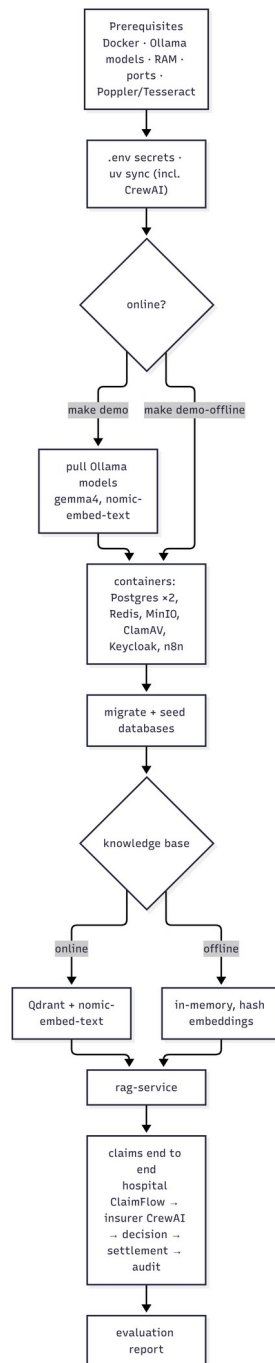


Figure 14: What `make demo` does, online and offline

| Command                        | What it does                                                             | Time                                     |
|--------------------------------|--------------------------------------------------------------------------|------------------------------------------|
| <code>make demo-check</code>   | Checks Docker, Ollama and models, RAM, free ports, Poppler and Tesseract | seconds                                  |
| <code>make demo-offline</code> | Whole pipeline with no model; in-memory knowledge base                   | 2-3 min                                  |
| <code>make demo</code>         | Whole pipeline on gemma4 with Qdrant and nomic-embed-text                | about 3 min per claim on an M-series Mac |
| <code>make demo-down</code>    | Stops background services and containers; data kept                      | seconds                                  |

Every step is idempotent and logs to `.e2e-logs/demo/<step>.log`; a failure is repeated after the summary with the last log lines. The prerequisites check names any program holding a needed port and gives the install command for missing tools.

## 14. Testing and evaluation

### 14.1 Automated tests

| Area                                                  | What is tested                                                                                                                |
|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Contract                                              | Models, state machines, HMAC, idempotency, audit chain                                                                        |
| hospital-api                                          | Auth and roles, cases, documents, completeness, router, claim builder, submission, queries, SSE (467 tests with the contract) |
| Crews                                                 | Hospital crew 58 tests, insurer crew 210 tests: agents, guardrails, flows, PII, injection corpus, timeouts, idempotency       |
| calc-engine                                           | Unit and property tests against hand-checked cases                                                                            |
| doc-pipeline, vision, RAG, simulators, synthetic data | Stage tests, detector evaluation, retrieval metrics, generator invariants                                                     |
| insurer-api                                           | Receipt, decision flow, gate, queries, golden end-to-end, security suite                                                      |
| UIs                                                   | Type check, lint, build; Playwright browser tests                                                                             |

### 14.2 End-to-end scenarios

| Scenario   | What it proves                                                                  |
|------------|---------------------------------------------------------------------------------|
| auto       | A clean claim is approved without a person, paid and audited                    |
| reviewer   | A claim above ₹50,000 waits for one approver                                    |
| dual       | A claim above ₹5,00,000 needs two approvers                                     |
| reject     | A reviewer rejects with a reason code; nothing is paid                          |
| queries    | Two answered query rounds; an unanswered third escalates to a senior            |
| callbacks  | Duplicate and out-of-order callbacks do not change a settled claim              |
| bank_retry | The bank fails the first payout; the insurer retries; the hospital ends settled |

### 14.3 Evaluation

Table 7: Evaluation on 300 synthetic cases (seed 42)

| Suite    | Metric                                                       | Value              | Target             | Pass |
|----------|--------------------------------------------------------------|--------------------|--------------------|------|
| calc     | Agreement with independent reference calculator              | 1.00               | 0.995              | Yes  |
| gate     | Decision route correctness                                   | 1.00               | 1.00               | Yes  |
| gate     | Wrongful automatic approvals                                 | 0.00               | 0.00               | Yes  |
| mapper   | Bill-line mapping accuracy                                   | 1.00               | 0.97               | Yes  |
| identity | Match recall / mismatch recall                               | 1.00 / 1.00        | 0.98 / 0.95        | Yes  |
| rag      | Recall@5 · MRR · nDCG@5                                      | 0.93 · 0.80 · 0.83 | 0.85 · 0.60 · 0.70 | Yes  |
| rag      | Citation precision                                           | 0.61               | 0.90               | No   |
| rag      | Insufficient-evidence · temporal-trap · numeric faithfulness | 1.00 · 1.00 · 1.00 | 0.90 · 0.95 · 1.00 | Yes  |

## 15. Results

### 15.1 Real-model run

`make demo` on a MacBook Air (Apple Silicon, Docker Desktop) with `gemma4` and `nomic-embed-text`: every step passed; the claim run (hospital ClaimFlow → submission → insurer CrewAI verification → automatic approval → settlement → audit) took 166 seconds.

```
=====
PASS Settings and packages (.env secrets, uv sync incl. CrewAI) 0s ready
PASS Ollama models 0s all models present: gemma4:latest, nomic-embed-text:latest
PASS Infrastructure containers (Postgres, Redis, MinIO, ClamAV, Keycloak) 12s up
PASS Insurer database container 6s up
PASS Hospital n8n (13 flows) 21s up
PASS Hospital database: migrate and seed demo users 1s done
PASS Qdrant vector database 22s up
PASS Knowledge base: policy wordings into Qdrant (nomic-embed-text) 7s seeded
PASS rag-service (policy wording search for both crews) 1s http://localhost:8400 (Qdrant, nomic-embed-text)
PASS Claims end to end: hospital ClaimFlow → insurer CrewAI agents → decision → settlement 166s FULL E2E PASSED
PASS Evaluation report 1s .e2e-logs/demo/eval/20261007T195240Z/report.md; below target
=====
DEMO PASSED (real local model); logs in .e2e-logs/demo
Next: 'make crew-demo' / 'make ins-crew-demo' (CrewAI flows alone), 'make crew-plot' (flow diagrams), 'make demo-down' (stop everything).
```

Figure 15: Summary of the real-model run on a MacBook Air (terminal output)

```
==> Evaluation report

# Evaluation 20261007T195240Z

cases=300 seed=42 llm_mode=offline-stub

| suite | metric | value | n | 95% CI | target | pass |
|-----|-----|-----|---|-----|-----|-----|
| calc | calc_agreement_vs_reference | 1.0 | 296 | (0.9872, 1.0) | 0.995 | True |
| gate | decision_route_correctness | 1.0 | 202 | (0.9813, 1.0) | 1.0 | True |
| gate | wrongful_auto_path | 0.0 | 202 | (-0.0, 0.0187) | 0.0 | True |
| mapper | mapper_rule_accuracy | 1.0 | 210 | (0.982, 1.0) | 0.97 | True |
| identity | identity_match_recall | 1.0 | 200 | (0.9812, 1.0) | 0.98 | True |
| identity | identity_mismatch_recall | 1.0 | 200 | (0.9812, 1.0) | 0.95 | True |
| rag | rag_recall@5 | 0.93 | 120 | None | 0.85 | True |
| rag | rag_mrr | 0.7977 | 120 | None | 0.6 | True |
| rag | rag_ndcg@5 | 0.8309 | 120 | None | 0.7 | True |
| rag | rag_citation_precision | 0.6104 | 120 | None | 0.9 | False |
| rag | rag_insufficient_evidence_accuracy | 1.0 | 120 | None | 0.9 | True |
| rag | rag_temporal_trap_accuracy | 1.0 | 120 | None | 0.95 | True |
| rag | rag_numeric_faithfulness | 1.0 | 120 | None | 1.0 | True |

## Not run (and why)

- **doc_parse_accuracy**:: needs doc-pipeline (Dev A) and rendered images
- **vision_stamp_signature**:: needs vision-service
- **pii_masking_recall**:: needs Presidio in doc-pipeline
- **verification_pipeline_e2e**:: run `pytest insurer/api/tests/test_golden_e2e.py` (needs Docker/Postgres)
- **llm_agent_quality**:: needs the live gateway (Gemini/Ollama); offline stubs only here
- **calibration**:: needs parser/vision confidences from the live pipeline
ok (1s) .e2e-logs/demo/eval/20261007T195240Z/report.md; below target: rag_citation_precision
```

Figure 16: Evaluation report as printed by the same run

### 15.2 Offline run

```
=====
PASS Settings and packages (.env secrets, uv sync incl. CrewAI) 0s ready
PASS Infrastructure containers (Postgres, Redis, MinIO, ClamAV, Keycloak) 18s up
PASS Insurer database container 1s up
PASS Hospital n8n (13 flows) 1s up
PASS Hospital database: migrate and seed demo users 1s done
PASS rag-service (policy wording search for both crews) 1s http://localhost:8400 (in-memory, hash embeddings)
PASS Claims end to end: hospital ClaimFlow → insurer CrewAI agents → decision → settlement 35s FULL E2E PASSED
PASS Evaluation report 1s .e2e-logs/demo/eval/20261007T194515Z/report.md; below target
=====
DEMO PASSED (offline: no model); logs in .e2e-logs/demo
Next: 'make crew-demo' / 'make ins-crew-demo' (CrewAI flows alone), 'make crew-plot' (flow diagrams), 'make demo-down' (stop everything).
```

Figure 17: Summary of the offline run on the same machine (claim run 35 s)

### 15.3 Observations

- **The estimate surfaces real disagreements.** For early-2026 stays the HEALTH-PLUS-GOLD wording in force (version 1) says 20% co-pay, while the insurer's rule set for that product says 0%: estimates read lower than the insurer pays. In practice this is exactly the kind of policy-configuration error a hospital wants to catch.
- **Deterministic overrides matter.** In offline runs the stand-in model reports no identity issue, yet the code's identity facts flag the mismatch and the claim goes to a person.
- **Portability.** Running on a Mac exposed issues that Linux hid (a removed MinIO image, an amd64-only ClamAV image, Docker Desktop host networking, missing PDF tools); all are fixed or checked by `make demo-check`.

## 16. Limitations

- **Citation precision 0.61** against 0.90, measured with offline stub embeddings; real embeddings may change it, and chunking and reranking need work.
- **Success metrics not yet measured:** manual baseline time, time to submit, queries per claim, manual steps saved, estimate accuracy against the insurer's amount.
- **Orchestration:** insurer-api still sequences verification (inline or via n8n) and calls each CrewAI agent per step; the CrewAI VerificationFlow is used by the demo, checks and evaluation.
- **Speed on CPU:** a real-model reply draft can take minutes on a laptop.
- **Synthetic data only:** stamp-detector and OCR numbers are measured on synthetic scans; real scans are untested.
- **Hospital Document and Validation crews** are still code services rather than CrewAI crews.

## 17. Future recommendations and roadmap

### 17.1 Short term (next 4-6 weeks)

- Add the missing success metrics: a timed manual baseline, time to submit, queries per claim, manual steps saved, and estimate vs final payable.
- Raise citation precision: section-aware chunking, stricter quote selection and a cross-encoder reranker; re-measure with real embeddings.
- Align the synthetic policy data (wording versions vs insurer rule sets) and add a consistency check.
- Let insurer-api run verification through the CrewAI VerificationFlow (`INS_ORCHESTRATOR=crew`), keeping the gate in code.

### 17.2 Medium term (3-6 months)

- Move document intake and validation into CrewAI crews with the existing rules as guardrails.
- Pre-authorisation flow and enhancement requests, using the same agents.
- GPU or hosted inference for speed, with masking kept in front of any non-local model.
- Pilot on de-identified real scans to calibrate OCR, classification and the stamp detector.

### 17.3 Long term

- Integrate with India's National Health Claims Exchange (NHCX) and ABDM health records instead of the simulators.
- Cross-claim fraud and anomaly analytics on the insurer side, with explainable signals.
- Multi-hospital, multi-insurer deployment with tenant isolation, monitoring dashboards and SLA reporting.
- Formal DPDP compliance review: consent records, retention schedules and data-principal requests.

## 18. Conclusion

The project shows that a claim can travel from raw scanned documents to settlement with AI agents doing the reading and writing, deterministic code doing the deciding, and people approving what matters. CrewAI gives the agents clear roles, guardrails and flows; n8n gives the processes visibility and control; the calculation engine and validation rules give exact, explainable money; and local models keep patient data on the machine. The system runs with one command, passes its scenarios and evaluation, and has a concrete roadmap for the remaining gaps.

## References

All links accessed October 2026.

[CrewAI documentation \(Agents, Tasks, Crews, Flows, guardrails\)](https://docs.crewai.com) <https://docs.crewai.com>  
[n8n documentation](https://docs.n8n.io) <https://docs.n8n.io>  
[Ollama](https://ollama.com) <https://ollama.com>  
[Gemma models \(Google\)](https://ai.google.dev/gemma) <https://ai.google.dev/gemma>  
[nomic-embed-text on Ollama](https://ollama.com/library/nomic-embed-text) <https://ollama.com/library/nomic-embed-text>  
[Qdrant documentation](https://qdrant.tech/documentation/) <https://qdrant.tech/documentation/>  
[FastAPI](https://fastapi.tiangolo.com) <https://fastapi.tiangolo.com>  
[Pydantic](https://docs.pydantic.dev) <https://docs.pydantic.dev>  
[SQLAlchemy](https://docs.sqlalchemy.org) <https://docs.sqlalchemy.org>  
[PostgreSQL 16](https://www.postgresql.org/docs/16/) <https://www.postgresql.org/docs/16/>  
[Microsoft Presidio \(PII detection and anonymisation\)](https://microsoft.github.io/presidio/) <https://microsoft.github.io/presidio/>  
[Tesseract OCR](https://github.com/tesseract-ocr/tesseract) <https://github.com/tesseract-ocr/tesseract>  
[Poppler PDF utilities](https://poppler.freedesktop.org) <https://poppler.freedesktop.org>  
[MinIO object storage \(source\)](https://github.com/minio/minio) <https://github.com/minio/minio>  
[Keycloak](https://www.keycloak.org/documentation) <https://www.keycloak.org/documentation>  
[ClamAV](https://docs.clamav.net) <https://docs.clamav.net>  
[Redis](https://redis.io/docs/) <https://redis.io/docs/>  
[Next.js](https://nextjs.org/docs) <https://nextjs.org/docs>  
[Docker Compose](https://docs.docker.com/compose/) <https://docs.docker.com/compose/>  
[uv \(Python packaging\)](https://docs.astral.sh/uv/) <https://docs.astral.sh/uv/>  
[Langfuse \(LLM tracing\)](https://langfuse.com/docs) <https://langfuse.com/docs>  
[Insurance Regulatory and Development Authority of India \(IRDAI\)](https://irdai.gov.in) <https://irdai.gov.in>  
[Digital Personal Data Protection Act, 2023 \(MeitY\)](https://www.meity.gov.in/data-protection-framework) <https://www.meity.gov.in/data-protection-framework>  
[Ayushman Bharat Digital Mission \(ABDM\) and NHCX](https://abdm.gov.in) <https://abdm.gov.in>  
[WHO ICD-10 classification](https://icd.who.int/browse10) <https://icd.who.int/browse10>

## Appendix A. Glossary

| Term                | Meaning                                                                                |
|---------------------|----------------------------------------------------------------------------------------|
| Cashless claim      | Hospital is paid directly by the insurer for covered costs                             |
| Reimbursement claim | Member pays the hospital and is repaid by the insurer                                  |
| TPA                 | Third-Party Administrator that processes claims for insurers                           |
| Sum insured (SI)    | Maximum the insurer pays in a policy year                                              |
| Room-rent cap       | Daily limit on room charges, usually % of SI; overruns trigger proportional deductions |
| Sub-limit           | Maximum payable for a specific procedure                                               |
| Co-payment          | Share of each admissible claim the member pays                                         |
| Admissible amount   | What the insurer is expected to pay after all policy rules                             |
| T_auto, T_four      | Approval thresholds: ₹50,000 (auto) and ₹5,00,000 (two approvers)                      |
| Guardrail           | A check on an agent's output; failures are sent back to the agent                      |
| RAG                 | Retrieval-augmented generation: answers grounded in retrieved documents                |
| HMAC                | Keyed hash that signs messages so they cannot be forged or altered                     |
| DPDP                | Digital Personal Data Protection Act, 2023 (India)                                     |

## Appendix B. Repository structure

```

contract/          shared API contract: models, enums, state machines, HMAC,
idempotency, audit chain
hospital/api/      hospital-api (FastAPI, Alembic migrations 0001-0023)
hospital/crew/     CrewAI ClaimFlow, QueryFlow, policy estimate (config/agents.yaml,
flows.py, team.py)
hospital/n8n/      13 generated workflows
hospital/ui/       Next.js desk / officer / admin
insurer/api/      insurer-api (receipt, verification engine, gate, queries,
settlement)
insurer/crew/     7 CrewAI agents, VerificationFlow, QueryFlow (crewai_team.py,
flows.py)
insurer/calc_engine/ deterministic payout calculator
insurer/n8n/      19 generated workflows
insurer/ui/       Next.js reviewer / approver / admin
services/         doc-pipeline, vision-service, rag-service, tpa-sim
data/             synthetic generator and evaluation harness
eval/            insurer-side evaluation (run_eval.py)
infra/            Docker Compose, Keycloak, MinIO, Redis, ClamAV, Postgres init
scripts/         demo.py (make demo), end-to-end runs, checks
docs/            plan, decisions, runbook, diagrams, images

```

## Appendix C. Command reference

| Command                                                                       | Purpose                                                                      |
|-------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <code>make demo / make demo-offline</code>                                    | Whole system end to end, with or without the model                           |
| <code>make demo-check / make demo-down</code>                                 | Prerequisites only / stop everything                                         |
| <code>make init</code>                                                        | Create <code>.env</code> with generated secrets; install all Python packages |
| <code>make up-infra</code>                                                    | Start Postgres (hospital), Redis, MinIO, ClamAV, Keycloak; create buckets    |
| <code>make up-insurer, make up-n8n, make up-insurer-n8n, make up-rag</code>   | Insurer database; hospital n8n; insurer n8n; Qdrant                          |
| <code>make migrate, make seed, make migrate-insurer, make seed-insurer</code> | Database schemas and demo data                                               |
| <code>make seed-kb, make run-rag</code>                                       | Load policy wordings into Qdrant; run rag-service                            |
| <code>make crew-demo, make ins-crew-demo, make crew-plot</code>               | CrewAI flows alone; flow diagrams                                            |
| <code>make check-crew</code>                                                  | Insurer agents on the real model against known answers                       |
| <code>make e2e-hospital, make e2e-full</code>                                 | Scripted end-to-end runs                                                     |
| <code>make test, make test-insurer</code>                                     | Test suites                                                                  |
| <code>make diagrams</code>                                                    | Render the Mermaid diagrams to PNG and SVG                                   |

## Appendix D. Agent service endpoints

| Service       | Endpoint                                                                                   | Purpose                       |
|---------------|--------------------------------------------------------------------------------------------|-------------------------------|
| hospital crew | POST /v1/jobs/claim-build, claim-repair                                                    | Run ClaimFlow                 |
| hospital crew | POST /v1/jobs/query-triage, query-draft                                                    | Run QueryFlow                 |
| hospital crew | GET /v1/jobs/{id}, GET /v1/agents, GET /v1/health                                          | Job state, agent list, health |
| insurer crew  | POST /v1/identity/analyze,<br>/v1/authenticity/analyze,<br>/v1/coverage/analyze            | Single agents                 |
| insurer crew  | POST /v1/calc/map-lines,<br>/v1/supervisor/summarize, /v1/query/draft,<br>/v1/query/triage | Single agents                 |
| insurer crew  | POST /v1/flows/verification, /v1/flows/query                                               | CrewAI Flows                  |

## Appendix E. More n8n workflow screenshots

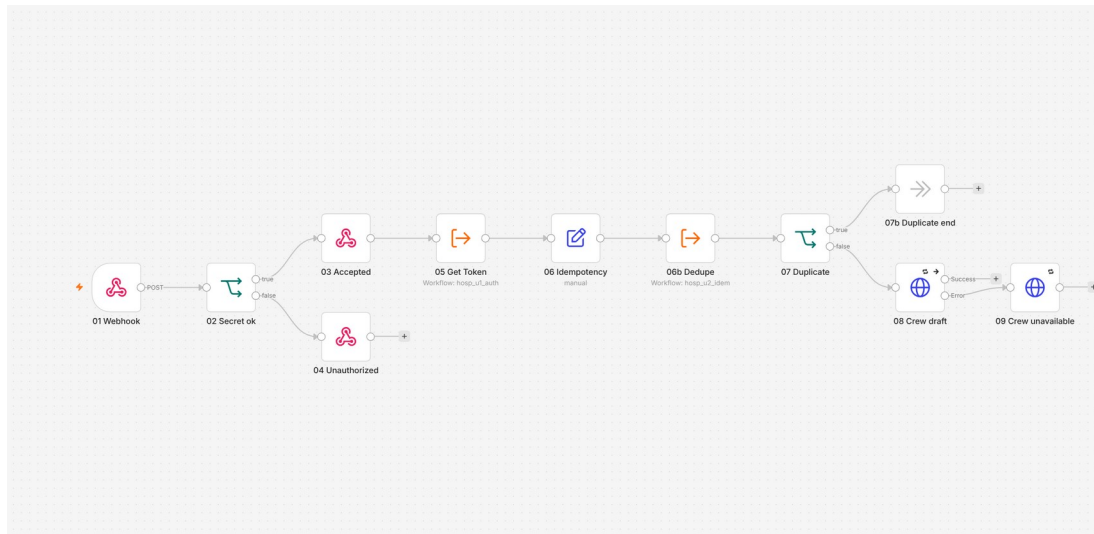


Figure 18: Hospital query-draft workflow

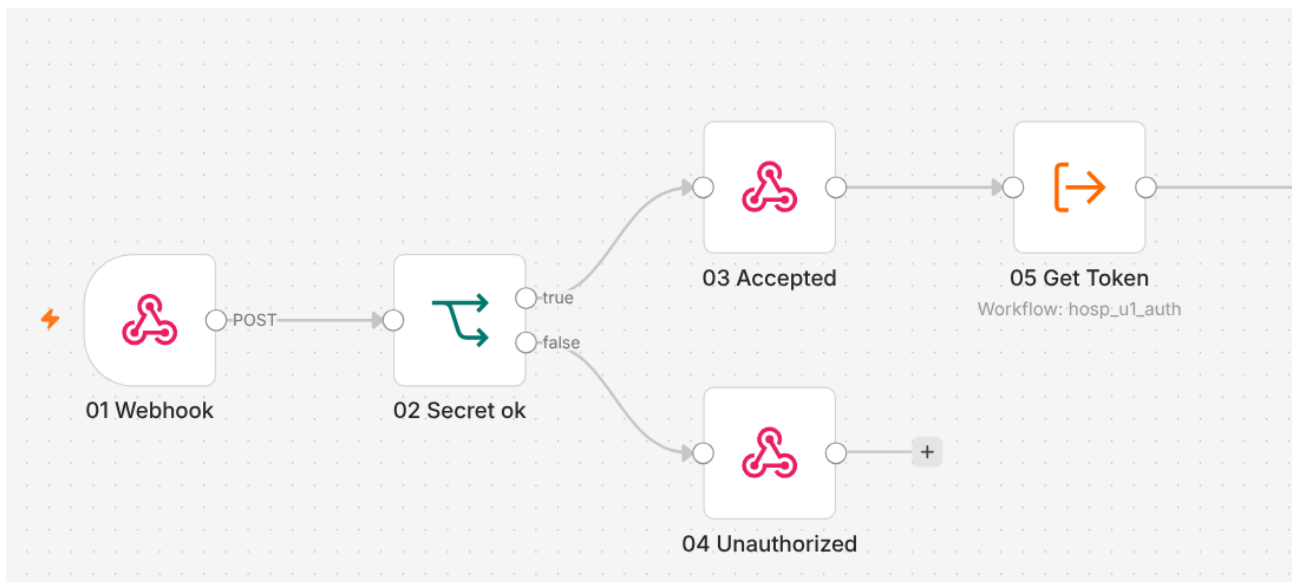


Figure 19: Hospital completeness loop (part 1)

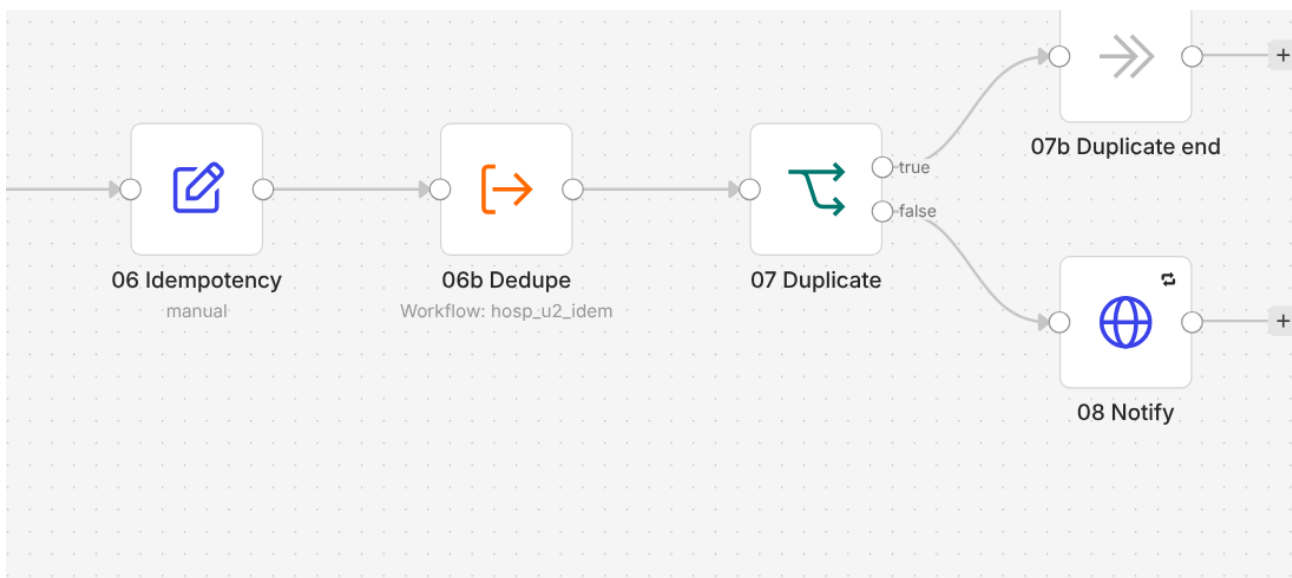


Figure 20: Hospital completeness loop (part 2)

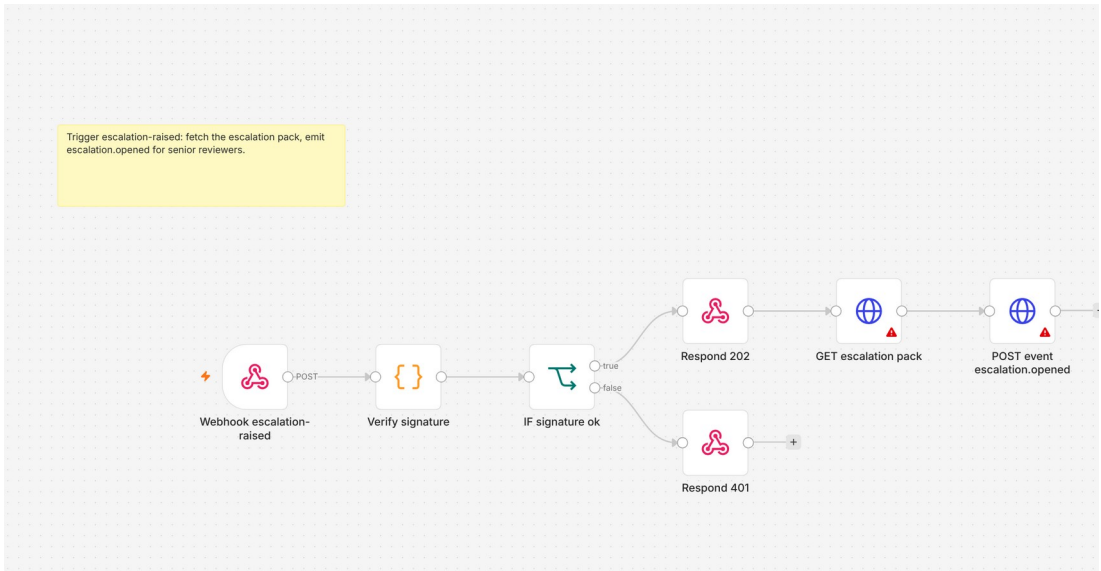


Figure 21: Insurer escalation workflow

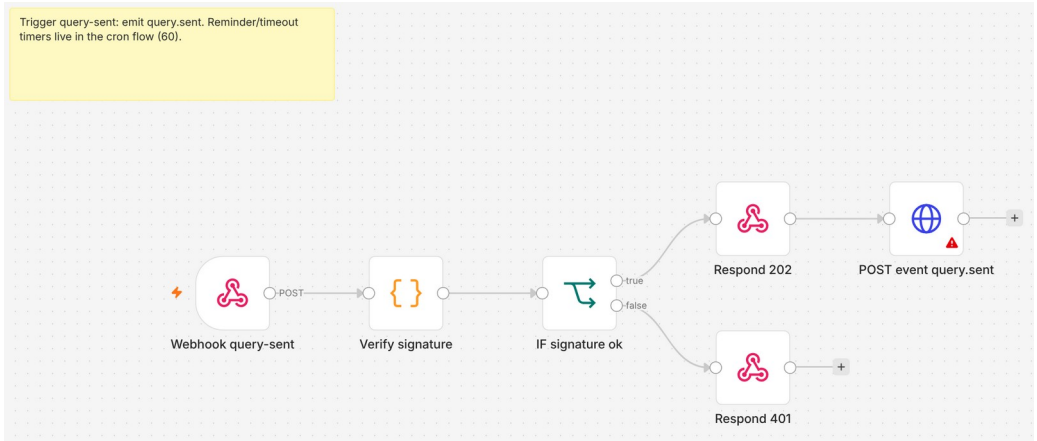


Figure 22: Insurer query-sent workflow

## Appendix F. Design decisions log (summary)

| Decision                                                               | Reason                                                   |
|------------------------------------------------------------------------|----------------------------------------------------------|
| CrewAI for all agents, behind the existing HTTP endpoints              | Course requirement; no change for APIs, n8n and UIs      |
| Agents without tools; deterministic tools in flow steps and guardrails | Small local models are unreliable at multi-step tool use |
| Ollama only, no provider keys                                          | Privacy and cost; DPDP-minded                            |
| Calculation engine as code, reused by the hospital                     | Exact, testable money on both sides                      |
| Qdrant instead of Pinecone, Langfuse instead of LangSmith              | Local and free; patient data stays on the machine        |
| MinIO built from source                                                | The public image was removed from Docker Hub             |
| Multi-arch ClamAV image                                                | Native on Apple Silicon                                  |